

Assignment-1

ELECTROMAGNETICS SIMULATIONS

EE25BTECH11011 - Navya Banavathu

1) 2D Electrostatic Simulation of a Parallel-Plate Capacitor

0.1 Theory for the Normal Case ($\kappa = 1$)

A parallel-plate capacitor with finite dimensions is simulated using a 2D grid. Due to finite size, fringing effects occur, so the problem is solved numerically.

In a charge-free region with uniform permittivity, the governing equation is:

$$\nabla^2 V = 0 \quad (1)$$

The computational domain is $5 \text{ mm} \times 5 \text{ mm}$, discretized into a 120×120 grid.

The plates are defined as:

- Top plate: $V = +5 \text{ V}$ at $y = +1.667 \text{ mm}$
- Bottom plate: $V = -5 \text{ V}$ at $y = -1.667 \text{ mm}$

All outer boundaries are grounded ($V = 0$).

Laplace's equation is solved using the finite difference method with Jacobi iteration:

$$V_{i,j} = \frac{1}{4} (V_{i+1,j} + V_{i-1,j} + V_{i,j+1} + V_{i,j-1}) \quad (2)$$

After convergence, the electric field is computed using:

$$\vec{E} = -\nabla V \quad (3)$$

$$E_x = -\frac{V_{i+1,j} - V_{i-1,j}}{2h}, \quad E_y = -\frac{V_{i,j+1} - V_{i,j-1}}{2h} \quad (4)$$

0.2 Python Code — Uniform Medium ($\kappa = 1$)

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import matplotlib.ticker as ticker
4
5 # GRID SETUP
6 N = 120
7 L = 5e-3
8 x = np.linspace(-L/2, L/2, N)
9 y = np.linspace(-L/2, L/2, N)
10 h = x[1] - x[0]
11
12 j_top = np.argmin(np.abs(y - 5e-3/3))
13 j_bot = np.argmin(np.abs(y - (-5e-3/3)))
14
15 V = np.zeros((N, N))
16 V[j_top, :] = +5.0
17 V[j_bot, :] = -5.0
18
19 # SOLVER -- vectorised Jacobi, 1000 iterations
20 for _ in range(1000):
21     V[1:-1, 1:-1] = 0.25 * (V[2:, 1:-1] + V[:-2, 1:-1] +
22                             V[1:-1, 2:] + V[1:-1, :-2])
23     V[j_top, :] = +5.0
24     V[j_bot, :] = -5.0
25
26 # ELECTRIC FIELD
27 dVdy, dVdx = np.gradient(V, h, h)
28 Ex = -dVdx
29 Ey = -dVdy
30 E_mag = np.sqrt(Ex**2 + Ey**2)
31
32 x_mm, y_mm = x * 1e3, y * 1e3
33 X_mm, Y_mm = np.meshgrid(x_mm, y_mm)
```

Grid setup, Jacobi solver, and electric field computation ($\kappa = 1$).

```
1 fig1, ax1 = plt.subplots(figsize=(7, 7))
2 ax1.streamplot(X_mm, Y_mm, Ex, Ey,
3               density=1.8, color='steelblue',
4               linewidth=0.9, arrowsize=1.2, arrowstyle='->')
5 ax1.plot([-2.5, 2.5], [y_mm[j_top]]*2, 'r-', lw=3,
6         label='Plate A  V = +5 V')
7 ax1.plot([-2.5, 2.5], [y_mm[j_bot]]*2, 'b-', lw=3,
8         label='Plate B  V = -5 V')
9 ax1.set_xlim(-2.5, 2.5); ax1.set_ylim(-2.5, 2.5)
10 ax1.set_xlabel('x (mm)', fontsize=12)
11 ax1.set_ylabel('y (mm)', fontsize=12)
12 ax1.set_title('Electric Field Lines -- Parallel-Plate Capacitor (k=1)',
13             fontsize=13)
14 ax1.legend(loc='upper right', fontsize=10)
15 ax1.set_aspect('equal')
16 ax1.grid(True, linestyle='--', linewidth=0.4, alpha=0.5)
```

```

17 plt.tight_layout()
18 plt.savefig('plot1_fieldlines.png', dpi=150, bbox_inches='tight')

```

Plot 1 — Electric field lines ($\kappa = 1$).

```

1 levels = np.linspace(-5, 5, 41)
2 fig2, ax2 = plt.subplots(figsize=(7, 7))
3 cf = ax2.contourf(X_mm, Y_mm, V, levels=levels, cmap='RdBu_r', alpha
4                   =0.85)
5 cs = ax2.contour(X_mm, Y_mm, V, levels=levels[::4],
6                  colors='k', linewidths=0.7, alpha=0.8)
7 ax2.clabel(cs, fmt='%+.1f V', fontsize=8, inline=True)
8 cbar = fig2.colorbar(cf, ax=ax2, fraction=0.046, pad=0.04)
9 cbar.set_label('Electric Potential V (volts)', fontsize=11)
10 cbar.ax.yaxis.set_major_formatter(ticker.FormatStrFormatter('%+.1f'))
11 ax2.plot([-2.5, 2.5], [y_mm[j_top]]*2, 'r-', lw=3, label='Plate A +5 V'
12          )
13 ax2.plot([-2.5, 2.5], [y_mm[j_bot]]*2, 'b-', lw=3, label='Plate B -5 V'
14          )
15 ax2.set_title('Equipotential Contour Map V(x,y) ( $\kappa=1$ )', fontsize=13)
16 ax2.set_aspect('equal')
17 plt.tight_layout()
18 plt.savefig('plot2_equipotential.png', dpi=150, bbox_inches='tight')

```

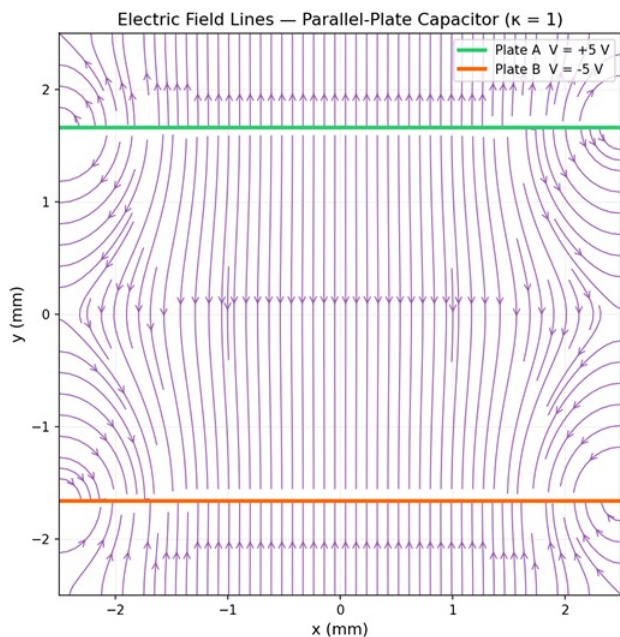
Plot 2 — Equipotential contour map ($\kappa = 1$).

```

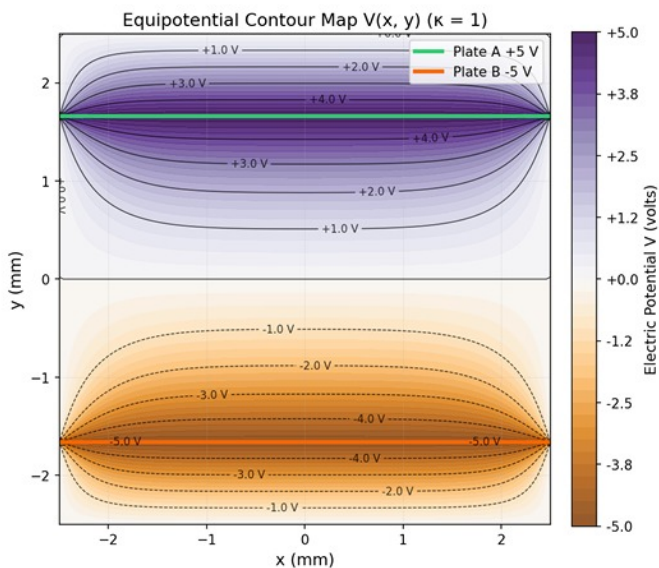
1 i_mid = np.argmin(np.abs(x))
2 E_vertical = E_mag[:, i_mid]
3
4 fig3, ax3 = plt.subplots(figsize=(8, 5))
5 ax3.plot(y_mm, E_vertical, color='steelblue', lw=1.8,
6          label='|E(x=0, y)|')
7 ax3.axvline(y_mm[j_top], color='red', ls='--', lw=1.2,
8             label=f'Plate A y = {y_mm[j_top]:.2f} mm')
9 ax3.axvline(y_mm[j_bot], color='blue', ls='--', lw=1.2,
10            label=f'Plate B y = {y_mm[j_bot]:.2f} mm')
11 ax3.axvspan(y_mm[j_bot], y_mm[j_top],
12             alpha=0.08, color='green', label='Inter-plate region')
13 ax3.set_xlabel('y (mm)', fontsize=12)
14 ax3.set_ylabel('|E| (V/m)', fontsize=12)
15 ax3.set_title('|E| Along Vertical Midline x=0 ( $\kappa=1$ )', fontsize=13)
16 ax3.legend(fontsize=10)
17 ax3.grid(True, linestyle='--', linewidth=0.4, alpha=0.6)
18 ax3.set_xlim(y_mm[0], y_mm[-1])
19 plt.tight_layout()
20 plt.savefig('plot3_Emag_vertical.png', dpi=150, bbox_inches='tight')

```

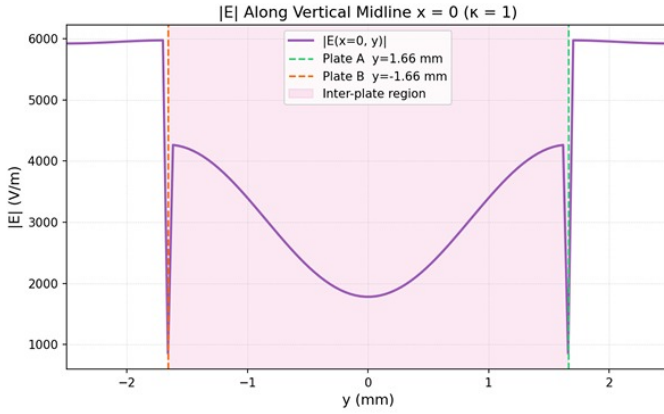
Plot 3 — $|E|$ along vertical midline $x = 0$ ($\kappa = 1$).



Equipotential contour map $V(x, y)$ for $\kappa = 1$.



Equipotential contour map $V(x, y)$ for $\kappa = 1$.



$|\vec{E}|$ along the vertical midline $x = 0$ for $\kappa = 1$.

0.3 Theory for the Dielectric Interface Case

In this case, the medium is non-uniform with different permittivities:

- $y > 0$: $\kappa_1 = 4$
- $y < 0$: $\kappa_2 = 1$

The governing equation becomes:

$$\nabla \cdot (\kappa \nabla V) = 0 \quad (5)$$

At the interface ($y = 0$), continuity conditions are applied:

- Tangential electric field is continuous
- Normal electric flux is continuous

This leads to a modified finite difference update at the interface:

$$V_{j_0,i} = \frac{V_{j_0-1,i} + 4V_{j_0+1,i} + 2.5(V_{j_0,i+1} + V_{j_0,i-1})}{10} \quad (6)$$

Physical Interpretation

- Electric field is weaker in the higher permittivity region ($y > 0$)
- Equipotential lines are denser in the lower permittivity region
- Field lines bend at the interface

0.4 Python Code — Dielectric Interface ($\kappa_1 = 4$, $\kappa_2 = 1$)

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import matplotlib.ticker as ticker
4
5 N = 120; L = 5e-3
6 x = np.linspace(-L/2, L/2, N)
7 y = np.linspace(-L/2, L/2, N)
8 h = x[1] - x[0]
9
10 kappa1 = 4.0
11 kappa2 = 1.0
12 kbar = (kappa1 + kappa2) / 2.0 # = 2.5
13 denom = kappa1 + kappa2 + 2.0 * kbar # = 10
14
15 j_top = np.argmin(np.abs(y - 5e-3/3))
16 j_bot = np.argmin(np.abs(y - (-5e-3/3)))
17 j_int = np.argmin(np.abs(y)) # interface row at y = 0
18
19 V = np.zeros((N, N))
20 V[j_top, :] = +5.0
21 V[j_bot, :] = -5.0
22
23 for _ in range(1000):
24     for j in range(1, N-1):
25         if j in (j_top, j_bot, j_int):
26             continue
27         V[j, 1:-1] = 0.25 * (V[j+1, 1:-1] + V[j-1, 1:-1] +
28                             V[j, 2:] + V[j, :-2])
29         # kappa-weighted stencil at interface row
30         V[j_int, 1:-1] = (kappa2 * V[j_int-1, 1:-1] +
31                             kappa1 * V[j_int+1, 1:-1] +
32                             kbar * (V[j_int, 2:] + V[j_int, :-2])) / denom
33         V[j_top, :] = +5.0
34         V[j_bot, :] = -5.0
35
36 # Electric field
37 dVdy, dVdx = np.gradient(V, h, h)
38 Ex = -dVdx; Ey = -dVdy
39 E_mag = np.sqrt(Ex**2 + Ey**2)
40 x_mm, y_mm = x*1e3, y*1e3
41 X_mm, Y_mm = np.meshgrid(x_mm, y_mm)
```

Dielectric-aware Jacobi solver with κ -weighted interface stencil.

```
1 from matplotlib.patches import Patch
2
3 # Plot 1: Field lines with dielectric regions shaded
4 fig1, ax1 = plt.subplots(figsize=(7, 7))
5 ax1.axhspan(0, 2.5, alpha=0.07, color='orange')
6 ax1.axhspan(-2.5, 0, alpha=0.07, color='cyan')
7 ax1.streamplot(X_mm, Y_mm, Ex, Ey, density=1.8, color='steelblue',
8               linewidth=0.9, arrowsize=1.2, arrowstyle='->')
```

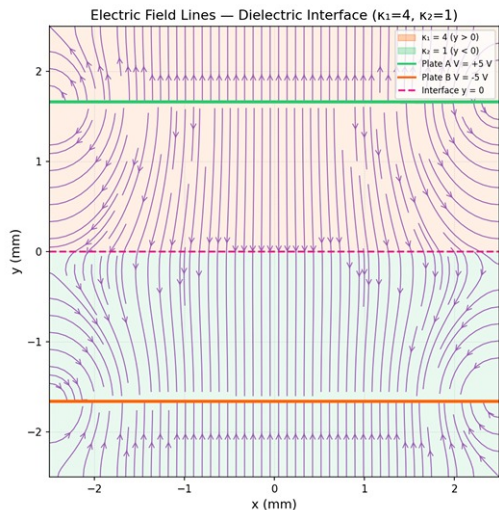
```

9 ax1.plot([-2.5,2.5],[y_mm[j_top]]*2,'r-',lw=3,label='Plate A V=+5V')
10 ax1.plot([-2.5,2.5],[y_mm[j_bot]]*2,'b-',lw=3,label='Plate B V=-5V')
11 ax1.axhline(0, color='k', ls='--', lw=1.2, label='Interface y=0')
12 ax1.set_title('Electric Field Lines -- Dielectric Interface (k1=4, k2
    =1)')
13 ax1.set_aspect('equal')
14 plt.tight_layout()
15 plt.savefig('diel_plot1_fieldlines.png', dpi=150, bbox_inches='tight')
16
17 # Plot 2: Equipotential with interface
18 levels = np.linspace(-5, 5, 41)
19 fig2, ax2 = plt.subplots(figsize=(7, 7))
20 cf = ax2.contourf(X_mm, Y_mm, V, levels=levels, cmap='RdBu_r', alpha
    =0.85)
21 cs = ax2.contour(X_mm, Y_mm, V, levels=levels[::4],
22                 colors='k', linewidths=0.7, alpha=0.8)
23 ax2.clabel(cs, fmt='%+.1f V', fontsize=8, inline=True)
24 fig2.colorbar(cf, ax=ax2, fraction=0.046, pad=0.04,
25              label='Electric Potential V (volts)')
26 ax2.axhline(0, color='k', ls='--', lw=1.2, label='Interface y=0')
27 ax2.set_title('Equipotential Map -- Dielectric Interface (k1=4, k2=1)')
28 ax2.set_aspect('equal')
29 plt.tight_layout()
30 plt.savefig('diel_plot2_equipotential.png', dpi=150, bbox_inches='tight
    ')
31
32 # Plot 3: |E| comparison -- dielectric vs uniform
33 # (requires running uniform solver first, see Listing 1)
34 V0 = np.zeros((N,N)); V0[j_top,:]=+5.0; V0[j_bot,:]=-5.0
35 for _ in range(1000):
36     V0[1:-1,1:-1]=0.25*(V0[2:,1:-1]+V0[:-2,1:-1]+
37                        V0[1:-1,2:]+V0[1:-1,-2])
38     V0[j_top,:]=+5.0; V0[j_bot,:]=-5.0
39 dVdy0,dVdx0=np.gradient(V0,h,h)
40 E_uni=np.sqrt(dVdx0**2+dVdy0**2)
41
42 i_mid=np.argmin(np.abs(x))
43 fig3,ax3=plt.subplots(figsize=(8,5))
44 ax3.plot(y_mm,E_mag[:,i_mid],color='darkorange',lw=1.8,
45         label='|E| dielectric (k1=4,k2=1)')
46 ax3.plot(y_mm,E_uni[:,i_mid], color='steelblue', lw=1.4, ls='--',
47         alpha=0.7, label='|E| uniform (k=1)')
48 ax3.axvline(0,color='k',ls=':',lw=1.0,label='Interface y=0')
49 ax3.axvspan(y_mm[j_bot],0,alpha=0.06,color='cyan',label='k2=1 region')
50 ax3.axvspan(0,y_mm[j_top],alpha=0.06,color='orange',label='k1=4 region'
    )
51 ax3.set_xlabel('y (mm)'); ax3.set_ylabel('|E| (V/m)')
52 ax3.set_title('|E| Along Midline x=0 -- Dielectric vs Uniform')
53 ax3.legend(fontsize=9,loc='upper left')
54 ax3.grid(True,linestyle='--',alpha=0.6)
55 plt.tight_layout()
56 plt.savefig('diel_plot3_comparison.png', dpi=150, bbox_inches='tight')

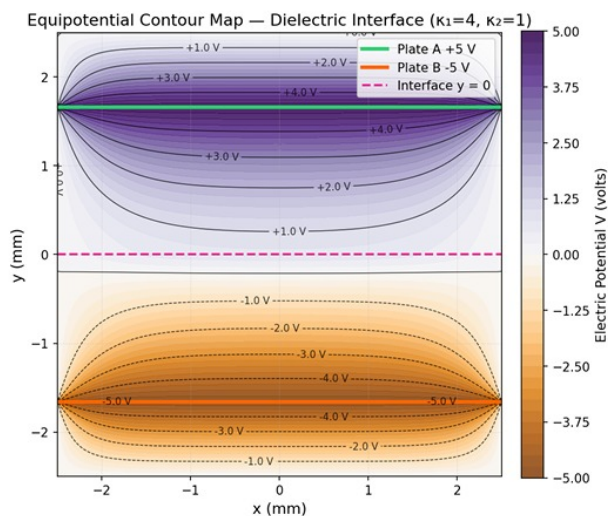
```

Dielectric case — field lines, equipotential, and comparison plots.

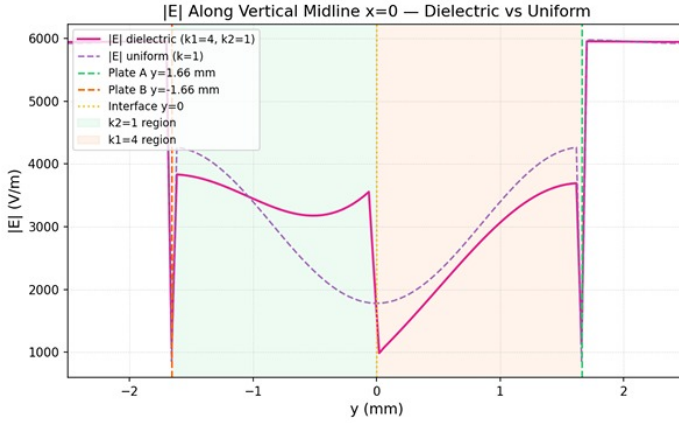
0.5 Results — Dielectric Interface ($\kappa_1 = 4, \kappa_2 = 1$)



Electric field lines with dielectric interface ($\kappa_1 = 4, \kappa_2 = 1$). Field lines bend at the interface due to change in permittivity.



Equipotential contour map with dielectric interface ($\kappa_1 = 4, \kappa_2 = 1$). Field is asymmetric; weaker in the high- κ region.



$|\vec{E}|$ along vertical midline $x = 0$: dielectric interface vs. uniform medium comparison. Field is asymmetric; weaker in high- κ region.

2) 2D Simulation of a Point Charge Near a Grounded Conducting Sphere

0.6 Theory

The interaction between a point charge and a grounded conducting sphere can be analyzed using the method of images.

A point charge Q placed at a distance d from the center of a sphere of radius R can be replaced by an image charge Q' placed inside the sphere such that the potential on the surface remains zero:

$$Q' = -\frac{R}{d}Q, \quad d' = \frac{R^2}{d} \quad (7)$$

The induced surface charge density is related to the electric field at the surface:

$$\sigma = -\epsilon_0 \frac{\partial V}{\partial r} \quad (8)$$

The total induced charge on the sphere is equal to the image charge:

$$Q_{\text{induced}} = Q' = -\frac{R}{d}Q \quad (9)$$

The potential at any point is given by:

$$V = \frac{1}{4\pi\epsilon_0} \left(\frac{Q}{r_1} + \frac{Q'}{r_2} \right) \quad (10)$$

0.7 Python Code — Method of Images

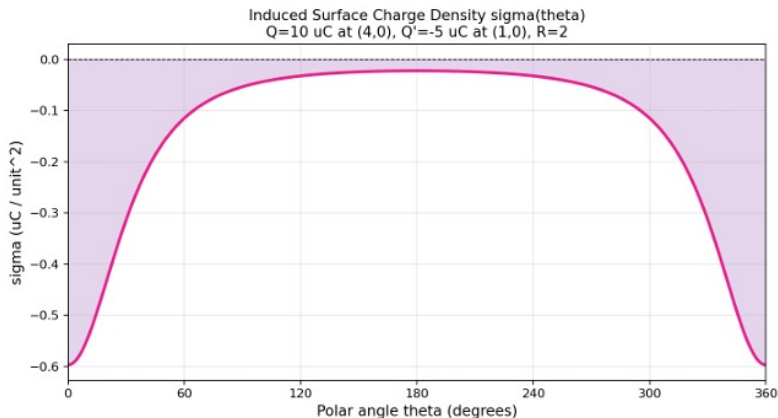
```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib.patches import Circle
4
5 R = 2.0; d = 4.0; Q = 10e-6; k = 8.99e9
6 Q_img = -R / d * Q          # = -5e-6 C
7 d_img = R**2 / d           # = 1.0 (image position)
8
9 # Grid
10 x = np.linspace(-6, 6, 600)
11 y = np.linspace(-6, 6, 600)
12 xx, yy = np.meshgrid(x, y)
13 inside = xx**2 + yy**2 < R**2
14
15 # Potential
16 r_Q = np.sqrt((xx - d)**2 + yy**2)
17 r_I = np.sqrt((xx - d_img)**2 + yy**2)
18 V = k*Q / r_Q + k*Q_img / r_I
19 V[inside] = np.nan
20
21 # Electric field (superposition)
22 Ex = k*Q*(xx-d) / r_Q**3 + k*Q_img*(xx-d_img) / r_I**3
23 Ey = k*Q*yy / r_Q**3 + k*Q_img*yy / r_I**3
24 Ex[inside] = 0; Ey[inside] = 0
25
26 # Surface charge density (full 360 degrees)
27 theta = np.linspace(0, 2*np.pi, 1000)
28 theta_deg = np.degrees(theta)
29 sig = -Q/(4*np.pi*R) * (d**2 - R**2) / \
30      (d**2 + R**2 - 2*d*R*np.cos(theta))**1.5
31
32 # --- Plot 1: sigma(theta) ---
33 fig, ax = plt.subplots(figsize=(9, 5))
34 ax.plot(theta_deg, sig*1e6, color='steelblue', lw=2.5)
35 ax.fill_between(theta_deg, sig*1e6, 0,
36                where=(sig < 0), alpha=0.25, color='steelblue')
37 ax.axhline(0, color='k', lw=0.8, ls='--')
38 ax.set_xlabel('Polar angle theta (degrees)', fontsize=12)
39 ax.set_ylabel('sigma (uC / unit^2)', fontsize=12)
40 ax.set_title("Induced Surface Charge Density sigma(theta)\n"
41             "Q=10 uC at (4,0), Q'=-5 uC at (1,0), R=2",
42             fontsize=12)
43 ax.set_xlim(0, 360); ax.set_xticks([0,60,120,180,240,300,360])
44 ax.grid(True, alpha=0.3)
45 plt.tight_layout()
46 plt.savefig('sphere_plot1_surface_charge.png', dpi=160,
47            bbox_inches='tight')
48
49 # --- Plot 2: Equipotential contours ---
50 fig2, ax2 = plt.subplots(figsize=(8, 8))
51 Vclip = np.clip(V, -3e5, 3e5)
52 cf = ax2.contourf(xx, yy, Vclip, levels=50,
53                  cmap='RdBu_r', alpha=0.85)
```

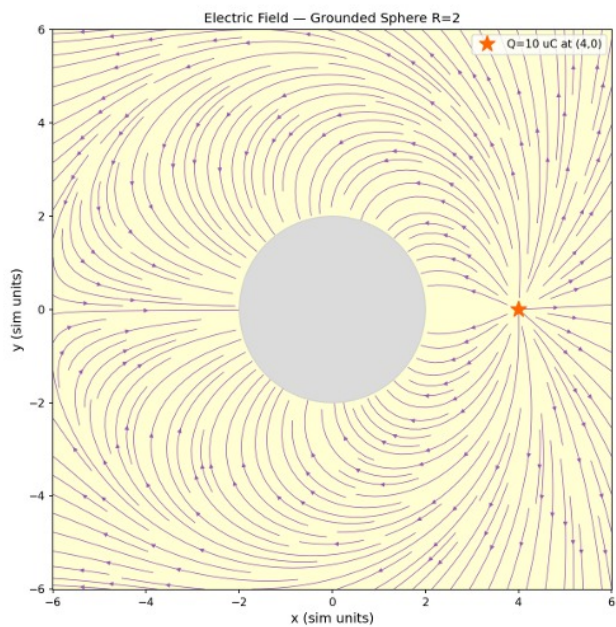
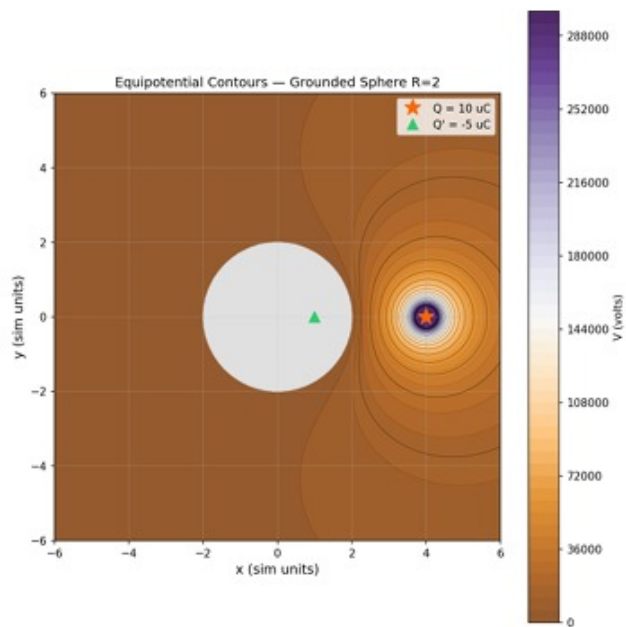
```

54 ax2.contour(xx, yy, Vclip, levels=20,
55             colors='k', linewidths=0.5, alpha=0.6)
56 plt.colorbar(cf, ax=ax2, label='V (volts)')
57 ax2.add_patch(Circle((0,0), R, color='gray', alpha=0.5, zorder=5))
58 ax2.plot(d, 0, 'r*', ms=15, zorder=10, label='Q = 10 uC')
59 ax2.plot(d_img, 0, 'b^', ms=10, zorder=10, label="Q' = -5 uC")
60 ax2.set_xlabel('x (sim units)'); ax2.set_ylabel('y (sim units)')
61 ax2.set_title('Equipotential Contours -- Grounded Sphere R=2')
62 ax2.set_aspect('equal'); ax2.legend(fontsize=10)
63 plt.tight_layout()
64 plt.savefig('sphere_plot2_equipotential.png', dpi=150,
65             bbox_inches='tight')
66
67 # --- Plot 3: Electric field lines ---
68 fig3, ax3 = plt.subplots(figsize=(8, 8))
69 E_mag_plot = np.where(inside, np.nan, np.sqrt(Ex**2+Ey**2))
70 ax3.pcolormesh(xx, yy, E_mag_plot, cmap='inferno',
71               shading='auto', alpha=0.8)
72 ax3.streamplot(x, y, Ex, Ey, density=2,
73               color='white', linewidth=0.7, arrowsize=0.8)
74 ax3.add_patch(Circle((0,0), R, color='white', alpha=0.6, zorder=5))
75 ax3.plot(d, 0, 'r*', ms=15, zorder=10, label='Q=10 uC at (4,0)')
76 ax3.set_xlabel('x (sim units)'); ax3.set_ylabel('y (sim units)')
77 ax3.set_title('Electric Field -- Grounded Sphere R=2')
78 ax3.set_aspect('equal'); ax3.legend(fontsize=10)
79 plt.tight_layout()
80 plt.savefig('sphere_plot3_fieldlines.png', dpi=150,
81             bbox_inches='tight')

```

Results — Point Charge and Grounded Sphere





0.8 Results and Observations

The induced charge density is maximum on the side of the sphere closest to the point charge and decreases away from it.

The equipotential contours are distorted near the sphere, showing the effect of induced charges.

Electric field lines originate from the point charge and terminate on the sphere surface, satisfying the grounded boundary condition.

3) Simulation of Electric Field Enhancement at a Sharp Conductor

Theory

At a conductor surface, the electric field is related to surface charge density:

$$\sigma = \epsilon_0 E_n \quad (11)$$

Charge accumulates more at sharp points (small radius of curvature), leading to a strong electric field near the tip:

$$|\mathbf{E}| \propto \frac{1}{\rho} \quad (12)$$

Numerical Model

The simulation is performed on a 100×100 grid.

Boundary conditions:

- Bottom: $V = 0$
- Top: $V = 100$
- Needle (center vertical line): $V = 0$

Laplace's equation is solved using the finite difference method:

$$V_{i,j} = \frac{1}{4} (V_{i-1,j} + V_{i+1,j} + V_{i,j-1} + V_{i,j+1}) \quad (13)$$

Electric field is computed as:

$$\vec{E} = -\nabla V \quad (14)$$

Python Code — Lightning Rod

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import matplotlib.ticker as ticker
4 from matplotlib.colors import LogNorm
5
6 N = 100
7 V = np.zeros((N, N))
8 needle_col = N // 2      # column 50 (horizontal centre)
9 needle_top = N // 2      # row 50 (tip of needle)
10 needle_bot = 0           # row 0 (grounded base)
11
12 def apply_boundaries(V):
13     V[0, :] = 0.0        # ground
14     V[-1, :] = 100.0     # cloud
15     V[needle_bot : needle_top + 1, needle_col] = 0.0 # needle
16
17 # Linear initial guess (speeds convergence ~2x)
18 for r in range(N):
19     V[r, :] = (r / (N - 1)) * 100.0
20 apply_boundaries(V)
```

Grid initialisation and boundary enforcement.

```
1 tol = 1e-3
2 max_iter = 10_000
3 convergence_hist = []
4
5 for it in range(max_iter):
6     V_old = V.copy()
7     for r in range(1, N - 1):
8         for c in range(1, N - 1):
9             if needle_bot <= r <= needle_top and c == needle_col:
10                 continue # skip conductor cells
11             V[r, c] = 0.25 * (V[r-1, c] + V[r+1, c] +
12                             V[r, c-1] + V[r, c+1])
13     apply_boundaries(V)
14     delta = np.max(np.abs(V - V_old))
15     convergence_hist.append(delta)
16     if delta < tol:
17         iters = it + 1
18         break
19
20 print(f"Converged in {iters} iterations (max DeltaV = {delta:.2e} V)")
21
22 # Electric field
23 Ey, Ex = np.gradient(-V)
24 E_mag = np.sqrt(Ex**2 + Ey**2)
```

Gauss–Seidel solver with convergence tracking.

```

1 import matplotlib.pyplot as plt
2 from matplotlib.colors import LogNorm
3 from mpl_toolkits.axes_grid1 import make_axes_locatable
4
5 # Plot 1: Convergence history
6 fig1, ax1 = plt.subplots(figsize=(8, 5))
7 ax1.semilogy(convergence_hist, color='steelblue', lw=1.5,
8              label='Max residual max|DeltaV|')
9 ax1.axhline(tol, color='gray', ls='--', lw=1.2,
10            label=f'Tolerance eps = {tol}')
11 ax1.scatter([iters-1],[convergence_hist[iters-1]],
12            color='orange',s=80,zorder=5,
13            label=f'Converged @ iter {iters}')
14 ax1.set_xlabel('Iteration'); ax1.set_ylabel('max|DeltaV| (V)')
15 ax1.set_title('Gauss-Seidel Convergence History')
16 ax1.legend(); ax1.grid(True, alpha=0.4)
17 plt.tight_layout()
18 plt.savefig('needle_plot1_convergence.png', dpi=150,
19            bbox_inches='tight')
20
21 # Plot 2: Potential heatmap + quiver
22 fig2, ax2 = plt.subplots(figsize=(9, 8))
23 im = ax2.imshow(V, origin='lower', cmap='plasma', aspect='auto')
24 plt.colorbar(im, ax=ax2, label='Electric Potential V (V)')
25 cs = ax2.contour(V, levels=np.arange(10,100,10),
26                 colors='white', linewidths=0.7, alpha=0.7)
27 ax2.clabel(cs, fmt='%d V', fontsize=7, inline=True)
28 step = 8
29 yi, xi = np.mgrid[0:N:step, 0:N:step]
30 ax2.quiver(xi, yi, Ex[:,::step,::step], Ey[:,::step,::step],
31           color='white', scale=80, alpha=0.7, width=0.003)
32 ax2.plot([needle_col,needle_col],[needle_bot,needle_top],
33         'lime', lw=3, label='Needle V=0')
34 ax2.plot(needle_col, needle_top, 'w', ms=10, zorder=5, label='Tip')
35 ax2.set_xlabel('x (grid cells)'); ax2.set_ylabel('y (grid cells)')
36 ax2.set_title('Potential Distribution & Electric Field E = -grad V')
37 ax2.legend(); plt.tight_layout()
38 plt.savefig('needle_plot2_potential.png', dpi=150,
39            bbox_inches='tight')
40
41 # Plot 3: Log |E| map
42 fig3, ax3 = plt.subplots(figsize=(9, 8))
43 E_safe = np.where(E_mag > 0, E_mag, 1e-10)
44 im3 = ax3.imshow(E_safe, origin='lower', cmap='inferno',
45                 norm=LogNorm(vmin=E_safe[E_safe>1e-5].min(),
46                             vmax=E_safe.max()),
47                 aspect='auto')
48 plt.colorbar(im3, ax=ax3, label='|E| (V/cell, log scale)')
49 circle = plt.Circle((needle_col, needle_top), 8,
50                    color='yellow', fill=False, ls='--', lw=1.5)
51 ax3.add_patch(circle)
52 ax3.plot([needle_col,needle_col],[needle_bot,needle_top],
53         'lime', lw=3, label='Needle')
54 tip_E = E_mag[needle_top+1, needle_col]
55 ax3.annotate(f'Max |E|\n= {tip_E:.2f} V/cell',
56            xy=(needle_col, needle_top+1),

```

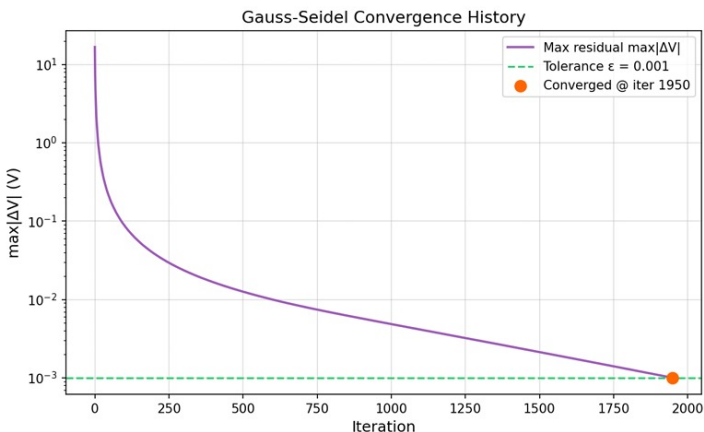
```

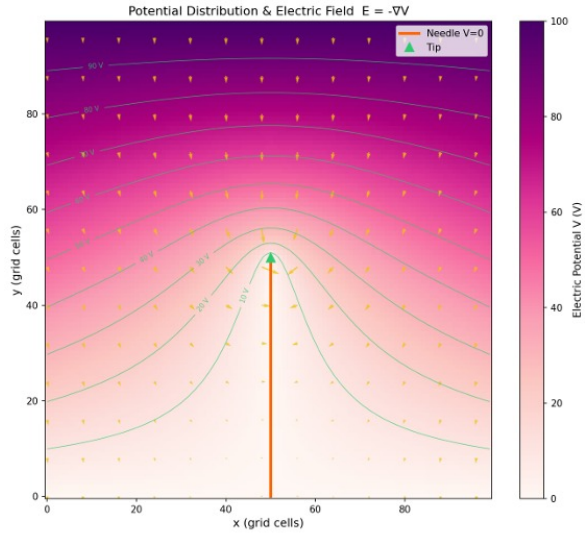
57         xytext=(needle_col+15, needle_top+10),
58         fontsize=9, color='white',
59         bbox=dict(boxstyle='round', fc='navy', alpha=0.7),
60         arrowprops=dict(arrowstyle='->', color='white'))
61 ax3.set_title('Electric Field Magnitude |E| (logarithmic)')
62 ax3.legend(); plt.tight_layout()
63 plt.savefig('needle_plot3_emag.png', dpi=150, bbox_inches='tight')
64
65 # Plot 4: Axial field profile above tip
66 above_tip = np.arange(needle_top+1, N)
67 dist      = above_tip - needle_top
68 E_axis    = E_mag[above_tip, needle_col]
69 ref       = E_axis[0] / dist**2
70
71 fig4, ax4 = plt.subplots(figsize=(8, 5))
72 ax4.plot(dist, E_axis, color='magenta', lw=1.8,
73         label='|E| along axis')
74 ax4.scatter([1],[E_axis[0]], color='orange', s=80, zorder=5,
75         label=f'Peak {E_axis[0]:.2f} V/cell')
76 ax4.plot(dist, ref, 'c--', lw=1.2, alpha=0.7,
77         label='~1/r^2 reference')
78 ax4.set_xlabel('Distance from needle tip (grid cells)')
79 ax4.set_ylabel('|E| (V/cell)')
80 ax4.set_title('Field Magnitude Along Needle Axis (above tip)')
81 ax4.legend(); ax4.grid(True, alpha=0.4); ax4.set_xlim(0, 50)
82 plt.tight_layout()
83 plt.savefig('needle_plot4_axial.png', dpi=150, bbox_inches='tight')

```

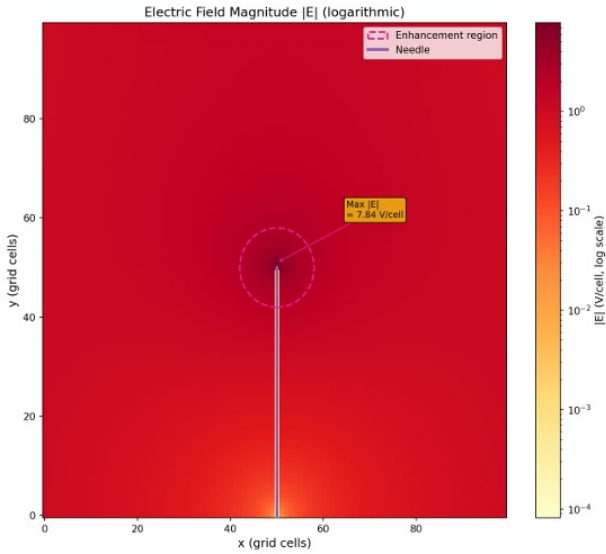
Four output plots for the lightning-rod simulation.

Results — Lightning Rod

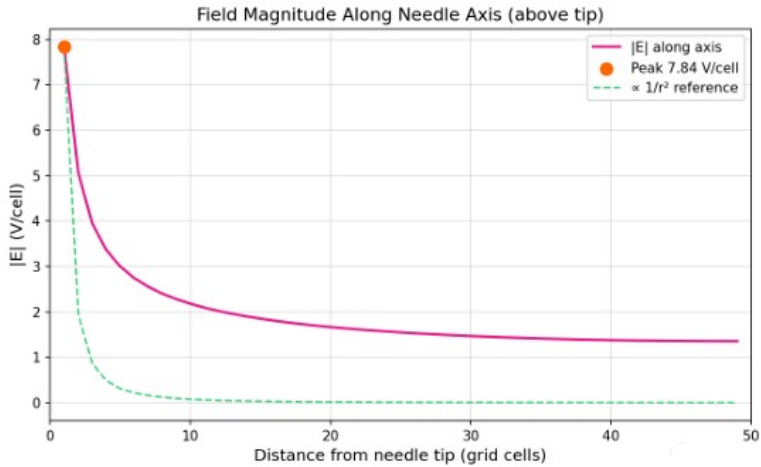




Potential heatmap $V(x, y)$. White contours are equipotentials at 10 V intervals; arrows show $\mathbf{E} = -\nabla V$. The white triangle marks the needle tip.



Logarithmic field magnitude map $\log_{10} |\mathbf{E}|$. The dashed circle marks the enhancement region at the needle tip.



$|E|$ along $x = 50$ from the tip to the cloud boundary. Peak at first cell above tip; consistent with $1/r^2$ point-charge decay.

Results and Observations

The electric field is maximum near the tip of the needle, as seen from the dense field vectors in that region. Away from the tip, the field becomes weaker and more uniform. This occurs due to charge accumulation at sharp points, leading to field enhancement. This explains the working of a lightning rod.